



廣東工業大學
GUANGDONG UNIVERSITY OF TECHNOLOGY

嵌入式系统课程设计报告

《基于 STM32F103C8T6 的智能手表设计》

姓 名： 温思迅

学 号： 3224009611

专 业： 微电子科学与工程

班 级： 微电子 2 班

队 名： 那就全队

队 友： 方鋆琪、冯宝倪

指导老师： 周贤中



集成电路学院

2026 年 7 月 13 日

摘要

本文介绍了嵌入式系统课程设计项目——基于 STM32F103C8T6 的智能手表设计。项目采用 C 语言进行嵌入式软件开发，以 STM32 微控制器作为核心控制单元，结合显示模块、传感器模块、按键交互模块以及电源管理模块，实现时间显示、数据采集、信息显示等智能手表基本功能。

系统在 STM32CubeIDE 开发环境下完成程序设计，通过软硬件结合的方式实现各模块协同工作，并在实际搭建的硬件平台上进行了功能测试与验证。项目展示了 STM32 微控制器在智能穿戴设备设计中的应用能力，为嵌入式系统综合开发提供了实践参考。

关键词：STM32；嵌入式系统；智能手表

目录

1	项目简介	1
1.1	项目背景	1
1.2	课题研究意义	1
1.3	系统功能	2
1.4	系统总体方案	2
1.5	开发环境	3
2	项目设计目标	4
3	项目设计与实现	5
3.1	硬件设计	5
3.1.1	系统硬件总体设计	5
3.2	STM32F103C8T6 主控模块	6
3.2.1	MPU6050 姿态传感器模块	6
3.2.2	OLED 显示模块	6
3.2.3	HC-05 蓝牙通信模块	7
3.2.4	旋转编码器模块	7
3.2.5	电源模块	8
3.3	软件设计	8
3.3.1	软件总体设计	8
3.3.2	STM32CubeMX 配置	9
3.3.3	FreeRTOS 任务设计	10
3.3.4	共享数据设计	10
3.3.5	MPU6050 数据采集	11
3.3.6	姿态解算	11
3.3.7	菜单控制设计	12
3.3.8	OLED 显示设计	12
3.3.9	蓝牙通信设计	13
3.3.10	计步算法设计	14
3.3.11	任务同步与互斥保护	14

3.4	代码介绍	15
3.4.1	初始化和配置	15
3.4.2	主功能实现	15
3.5	本章小结	16
4	功能展示	17
4.1	功能描述	17
4.2	演示结果	17
5	物料与成本	20
5.1	物料清单	20
5.2	项目总成本	20
5.3	损耗与损坏记录	20
6	个人贡献	21
6.1	报告撰写	21
6.2	编码工作	21
6.3	硬件连接	21
6.4	开发问题描述与解决方案	21
6.4.1	FreeRTOS 移植调试	21
6.4.2	MPU6050 姿态数据显示调试	22
6.4.3	菜单功能调试	22
6.4.4	蓝牙通信调试	23
7	项目成果与反思	24
7.1	成果展示	24
7.2	项目评估	25
7.2.1	成功之处	25
7.2.2	不足之处	25
7.3	个人反思	25
A	设计图纸	27
B	代码清单	27

1 项目简介

1.1 项目背景

近年来，随着嵌入式系统、物联网技术以及智能可穿戴设备的快速发展，智能手表逐渐成为消费电子产品的重要组成部分。与传统电子手表相比，智能手表不仅能够显示时间，还集成了运动检测、姿态识别、无线通信、健康监测等多种功能，在运动健身、医疗健康以及智能家居等领域具有广泛的应用前景。

近年来，ARM Cortex-M 系列微控制器凭借功耗低、性能高、开发资料丰富等优点，已成为嵌入式开发的重要平台。其中 STM32F103 系列采用 ARM Cortex-M3 内核，主频最高可达 72MHz，片上集成丰富的 GPIO、USART、I²C、SPI、DMA 以及定时器等资源，能够满足绝大多数智能终端的开发需求，因此成为高校嵌入式课程设计的主要开发平台。

与此同时，随着系统功能不断增加，传统裸机程序逐渐暴露出代码耦合度高、可维护性差、实时性不足等问题。FreeRTOS 作为目前应用最广泛的轻量级实时操作系统之一，能够实现任务调度、任务同步、互斥锁、消息队列等功能，使复杂嵌入式系统具有更好的实时性能和可扩展性。

基于上述背景，本课题设计并实现了一款基于 STM32F103C8T6 的智能手表系统，实现姿态检测、OLED 显示、蓝牙通信、运动计步以及菜单交互等功能，通过项目实践掌握 STM32 嵌入式开发流程以及 FreeRTOS 实时操作系统的应用。

1.2 课题研究意义

智能穿戴设备是近年来嵌入式系统的重要研究方向，其涉及传感器采集、无线通信、人机交互、实时操作系统等多个知识领域，具有较高的综合实践价值。

本项目通过完成智能手表设计，实现了多个典型嵌入式技术的综合应用，包括：

- STM32 微控制器外设驱动开发；
- I²C 总线通信及 MPU6050 驱动；
- OLED 图形界面设计；
- FreeRTOS 多任务调度；
- DMA 串口通信；

- 蓝牙无线数据传输；
- 姿态解算算法；
- 简单运动计步算法。

在项目开发过程中，需要解决硬件接口连接、多任务同步、共享数据保护以及系统调试等问题，对于提高嵌入式系统综合开发能力具有重要意义。

1.3 系统功能

本智能手表主要实现以下功能：

1. 时间显示功能；
2. MPU6050 姿态检测；
3. Pitch、Roll 姿态角实时计算；
4. OLED 实时数据显示；
5. 旋转编码器菜单切换；
6. HC-05 蓝牙通信；
7. 手机实时显示姿态数据；
8. 运动计步；
9. FreeRTOS 多任务运行。

1.4 系统总体方案

本系统采用模块化设计，各模块之间相互独立，并通过共享数据结构进行数据交换。整个系统主要由 STM32 主控制器、MPU6050 姿态传感器、OLED 显示模块、HC-05 蓝牙模块、旋转编码器输入模块以及软件系统组成。

系统总体结构如图1所示。

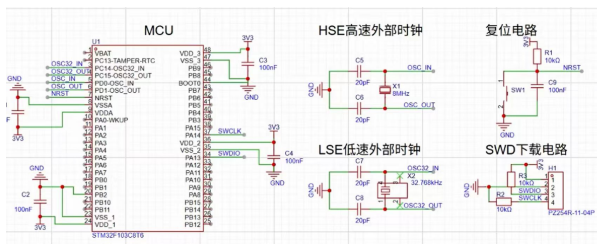


图 1 系统总体结构框图

系统工作流程如下：

MPU6050 采集三轴加速度和角速度数据，通过 I²C 总线发送至 STM32 微控制器；STM32 完成姿态解算、计步计算以及数据处理；OLED 实时显示当前页面内容；蓝牙模块通过 USART 接口向手机发送姿态角和步数等数据；旋转编码器用于菜单切换，实现良好的用户交互。

1.5 开发环境

本项目的软件开发环境如下：

表 1 开发环境

项目	软件/硬件
MCU	STM32F103C8T6
IDE	STM32CubeIDE
配置工具	STM32CubeMX
RTOS	FreeRTOS
调试器	ST-Link V2
姿态传感器	MPU6050
显示模块	OLED
蓝牙模块	HC-05
编程语言	C 语言

2 项目设计目标

随着智能穿戴设备的快速发展，集运动监测、健康管理和无线通信于一体的智能手表逐渐成为嵌入式系统的重要应用方向。传统手表仅具备时间显示功能，难以满足现代用户对运动记录、姿态监测以及信息交互等方面的需求。基于嵌入式微控制器设计智能手表，不仅能够实现多种传感器数据采集与处理，还能够结合显示、通信及人机交互等技术，为用户提供更加丰富和智能的使用体验。

STM32 系列微控制器具有较高的运算性能、丰富的外设资源以及较低的功耗，能够满足智能手表对实时性、可靠性和功能扩展性的要求。本项目结合嵌入式系统课程所学知识，选用 STM32F103C8T6 作为核心控制器，结合 MPU6050 姿态传感器、OLED 显示屏、HC-05 蓝牙模块以及旋转编码器等硬件，设计并实现一款具有姿态检测、运动计步、数据显示和无线通信等功能的智能手表系统，将课堂理论知识应用于实际工程开发，提高嵌入式系统综合设计能力。

本项目旨在设计并实现一款功能完善、运行稳定的智能手表系统，主要目标包括：

- **实现姿态检测功能：**利用 MPU6050 六轴惯性传感器实时采集三轴加速度和三轴角速度数据，通过姿态解算算法计算 Pitch 角和 Roll 角，实现设备姿态实时监测。
- **实现运动计步功能：**基于加速度数据设计计步算法，对人体步行过程进行识别，实现运动步数实时统计与显示。
- **开发人机交互界面：**利用 OLED 显示模块设计多页面用户界面，并结合旋转编码器实现菜单切换和功能选择，提高系统交互体验。
- **实现蓝牙无线通信：**采用 HC-05 蓝牙模块完成智能手表与手机之间的数据通信，实现姿态角、步数等信息的实时发送，为后续移动端应用提供数据支持。
- **完成 FreeRTOS 多任务系统设计：**将传感器采集、OLED 显示、菜单控制、蓝牙通信等功能划分为独立任务，利用 FreeRTOS 实现多任务调度，并通过互斥锁保护共享数据，提高系统实时性和稳定性。
- **完成系统集成与功能验证：**在 STM32F103C8T6 开发平台上完成硬件搭建和软件开发，对姿态检测、OLED 显示、菜单控制、蓝牙通信以及计步功能进行综合调试与测试，验证系统整体性能和可靠性。

3 项目设计与实现

3.1 硬件设计

3.1.1 系统硬件总体设计

本系统采用模块化设计思想，以 STM32F103C8T6 作为核心控制器，各功能模块之间通过标准总线接口进行连接，实现姿态检测、数据显示、无线通信以及人机交互等功能。

整个硬件系统主要由以下几个部分组成：

- STM32F103C8T6 最小系统
- MPU6050 六轴姿态传感器
- OLED 显示模块
- HC-05 蓝牙通信模块
- 旋转编码器输入模块
- 电源模块

系统总体硬件结构如图2所示。

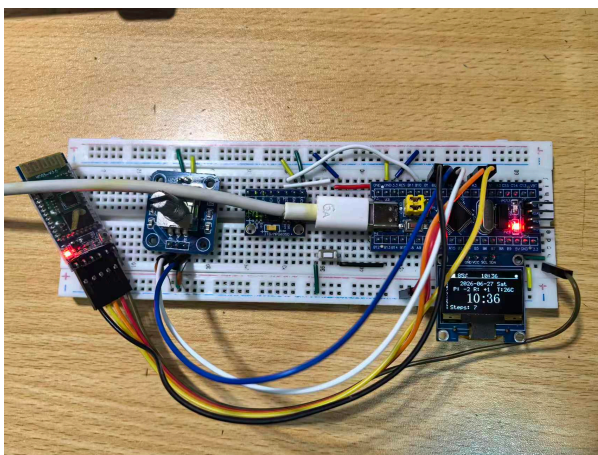


图 2 系统硬件图

STM32 作为整个系统的控制核心，通过 I²C 总线与 MPU6050 和 OLED 模块通信，通过 USART 接口连接 HC-05 蓝牙模块，通过 TIM3 读取旋转编码器，实现整个智能手表的数据采集、处理及显示。

3.2 STM32F103C8T6 主控模块

本系统选用意法半导体 (STMicroelectronics) 生产的 STM32F103C8T6 微控制器作为主控制器。

STM32F103C8T6 采用 ARM Cortex-M3 内核, 最高主频 72MHz, 具有较高的数据处理能力, 同时片上集成丰富的外设资源, 非常适合智能终端开发。

STM32 主要负责完成以下工作:

1. 初始化各外设;
2. 周期采集 MPU6050 数据;
3. 完成姿态解算;
4. 执行计步算法;
5. 更新 OLED 显示;
6. 蓝牙通信;
7. FreeRTOS 任务调度。

3.2.1 MPU6050 姿态传感器模块

MPU6050 是一款集成三轴加速度计和三轴陀螺仪的六轴惯性测量单元 (IMU), 能够实时输出三轴加速度、角速度以及芯片内部温度数据。

本项目采用 I²C 总线进行通信。

系统通过周期读取 MPU6050 输出数据, 计算 Pitch 角和 Roll 角, 并将结果发送至 OLED 显示及蓝牙模块。

3.2.2 OLED 显示模块

本项目采用 0.96 英寸 OLED 显示屏, 分辨率为 128×64 像素, 驱动芯片采用 SSD1306。

OLED 具有功耗低、响应速度快、显示清晰等特点, 非常适合作为智能手表的人机交互界面。

OLED 同样采用 I²C 接口与 STM32 通信。

OLED 主要显示内容包括:

- 时间

- 姿态角（Pitch、Roll）
- 加速度
- 陀螺仪数据
- 温度
- 步数
- 蓝牙连接状态
- 系统菜单

为了方便用户操作，本项目设计了多页面 UI 界面，通过旋转编码器切换不同页面，实现较好的交互体验。

3.2.3 HC-05 蓝牙通信模块

为了实现智能手表与手机之间的数据交互，本系统采用 HC-05 蓝牙串口模块。

HC-05 支持 Bluetooth 2.0 协议，通过 USART 串口与 STM32 连接。

系统采用 DMA 方式完成串口发送，提高 CPU 利用率。

本项目蓝牙主要实现以下功能：

1. 手机连接智能手表；
2. 实时发送 Pitch 角；
3. 实时发送 Roll 角；
4. 实时发送运动步数；

为了保证数据传输可靠性，本系统设计了简单的数据帧格式，包括帧头、命令字、数据长度、数据区、校验位以及帧尾。

3.2.4 旋转编码器模块

为了实现菜单操作，本系统采用机械旋转编码器作为输入设备。

编码器 A、B 两相信号连接至 STM32 的 TIM3，配置为 Encoder Mode，实现硬件计数。

系统根据 TIM3 计数值变化判断旋转方向，并设置每 4 个计数切换一页菜单，提高菜单操作稳定性，避免由于机械抖动导致误翻页。

此外，编码器按键预留菜单确认功能，为后续功能扩展提供接口。

3.2.5 电源模块

整个系统采用 5V USB 供电，经开发板稳压电路转换为 3.3V，为 STM32、MPU6050 以及 OLED 模块供电。

HC-05 采用 5V 供电，其串口通信电平与 STM32 兼容。

为了保证系统稳定运行，各模块电源端均设置去耦电容，以降低电源纹波，提高系统抗干扰能力。

3.3 软件设计

3.3.1 软件总体设计

为了提高系统的实时性、稳定性以及后续功能扩展能力，本项目采用 FreeRTOS 实时操作系统作为软件平台，对各功能模块进行任务划分。相比传统裸机程序，FreeRTOS 能够有效管理多个功能模块之间的执行关系，使各任务按照固定周期运行，提高了系统资源利用率。

本项目采用 FreeRTOS 实时操作系统，将整个系统划分为多个独立任务。

主要包括：

- Timekeep Task

时间维护任务，每秒更新一次系统时间

- Sensor Task

负责读取 MPU6050 数据，完成姿态解算以及计步算法。

- Display Task

负责 OLED 刷新，根据当前菜单页面显示对应内容。

- Menu Task

负责读取旋转编码器，实现菜单切换。

- Bluetooth Task

负责检测蓝牙连接状态，并实时发送姿态角及步数。

为了避免多个任务同时访问共享数据，本系统采用 FreeRTOS 互斥锁（Mutex）保护全局共享结构体 `watch_data`，提高系统稳定性。

整个系统主要完成以下几个功能：

- 系统初始化
- 传感器数据采集
- 姿态角计算
- OLED 数据显示
- 菜单控制
- 蓝牙通信
- 运动计步

各模块之间采用共享数据结构进行通信，并利用 FreeRTOS 互斥锁保护共享资源，避免多个任务同时访问导致的数据错误。

3.3.2 STM32CubeMX 配置

为了提高开发效率，本项目采用 STM32CubeMX 完成外设初始化配置，并生成工程框架。

表 2 模块配置表

模块	配置方式	用途
I2C1	Fast Mode	OLED 通信
I2C2	Fast Mode	MPU6050 通信
USART1	DMA	HC-05 蓝牙通信
TIM2	Base Timer	系统定时
TIM3	Encoder Mode	菜单编码器
DMA	Circular + Normal	蓝牙收发

(续表)

模块	配置方式	用途
GPIO	Input/Output	按键及状态检测
FreeRTOS	CMSIS V1	多任务管理

CubeMX 完成底层初始化后，应用程序主要在 FreeRTOS 任务中运行。

3.3.3 FreeRTOS 任务设计

根据系统功能，将整个程序划分为多个独立任务，各任务之间互不影响。

任务名称	周期	功能
Sensor Task	100ms	MPU6050 采集、姿态解算、计步
Display Task	100ms	OLED 刷新显示
Menu Task	20ms	编码器菜单切换
Bluetooth Task	100ms	蓝牙发送姿态角和步数

采用多任务设计后，各功能能够并行运行，提高了系统实时性能，同时便于后续增加新的功能模块。

3.3.4 共享数据设计

由于多个任务均需要访问姿态角、步数以及时间等数据，因此系统设计了共享数据结构 SmartWatchData。

主要包括：

- 加速度数据
- 陀螺仪数据
- Pitch 角
- Roll 角
- 温度
- 步数
- 时间信息

- IMU 状态

为了避免多个任务同时修改共享数据，本项目采用 FreeRTOS 互斥锁（Mutex）进行保护。

Sensor Task 负责更新共享数据，其余任务仅负责读取数据。

3.3.5 MPU6050 数据采集

MPU6050 通过 I2C 总线与 STM32 通信。

Sensor Task 每 100ms 完成一次数据采集。

主要流程如下：

1. 初始化 MPU6050；
2. 读取三轴加速度；
3. 读取三轴角速度；
4. 读取温度；
5. 更新共享数据。

为了提高系统稳定性，当 I2C 通信失败时，将 IMU 状态置为异常，并等待下一次采样重新读取。

3.3.6 姿态解算

MPU6050 输出的数据包括加速度和角速度。

为了得到稳定的姿态角，本系统采用互补滤波算法，将加速度数据和陀螺仪积分结果进行融合，计算 Pitch 角和 Roll 角。

姿态计算完成后，将结果保存至共享数据结构：

- Pitch
- Roll

随后 OLED 显示任务和蓝牙任务均可读取姿态角进行显示与发送。

3.3.7 菜单控制设计

为了实现良好的人机交互，本系统采用机械旋转编码器作为输入设备，并利用 STM32 定时器 TIM3 的 Encoder Mode 实现硬件计数。

编码器 A、B 两相信号分别连接至 TIM3 输入通道，STM32 自动完成正交解码，无需软件进行复杂的相位判断。当用户旋转编码器时，TIM3 计数值自动增加或减少，软件仅需读取当前计数器的值即可判断旋转方向。

菜单任务 (Menu Task) 周期为 20ms，每次读取 TIM3 当前计数值，并计算与上一次计数值之间的差值。当累计计数达到 4 个脉冲时，执行一次菜单翻页操作，以减少机械编码器抖动造成的误触发，提高菜单切换稳定性。

菜单页面采用循环切换方式，当超过最后一页时自动返回第一页，当向前切换到第一页之前时则返回最后一页，实现连续循环浏览。

系统目前共设计四个页面：

1. 时钟主页；
2. IMU 数据显示页面；
3. 蓝牙状态页面；
4. 系统信息页面。

菜单任务仅负责修改当前页面编号，而 OLED 显示任务根据当前页面编号调用对应的绘制函数，实现菜单显示与业务逻辑分离，提高了程序可维护性。

3.3.8 OLED 显示设计

OLED 显示模块采用 SSD1306 驱动芯片，分辨率为 128×64 像素，通过 I²C 总线与 STM32 通信。

为了提高刷新效率，本系统采用缓存区绘制方式。程序首先在 RAM 缓存中完成整个界面的绘制，随后调用 OLED_Update() 函数一次性刷新至 OLED 显示屏，避免频繁写屏造成闪烁现象。

系统根据不同菜单页面设计了不同的人机交互界面。

其中主页面用于显示当前时间和系统状态；IMU 页面显示三轴加速度、三轴角速度、Pitch 角、Roll 角、温度以及运动步数；蓝牙页面显示蓝牙连接状态及数据发送情况；系统信息页面用于显示系统版本等基本信息。

在姿态显示页面中，还设计了姿态指示器。根据 Pitch 角和 Roll 角计算小球在圆形坐标中的位置，并实时绘制姿态变化，使用户能够更加直观地观察设备姿态变化情况。

3.3.9 蓝牙通信设计

为了实现智能手表与手机之间的数据交互，本系统采用 HC-05 蓝牙模块进行无线通信。

HC-05 通过 USART1 与 STM32 连接，串口参数设置如下：

表 3 蓝牙串口配置

参数	配置
波特率	9600bps
数据位	8bit
停止位	1bit
校验位	None
流控制	None
发送方式	DMA
接收方式	DMA 循环接收

为了降低 CPU 占用率，系统采用 DMA 方式完成串口数据发送，并利用 UART 发送完成中断释放发送标志位，实现连续稳定的数据发送。

Bluetooth Task 周期为 100ms，每次运行时首先检测蓝牙连接状态。当 HC-05 连接手机后，读取共享数据结构中的姿态角及步数，并发送至手机。

为了保证通信可靠性，本项目设计了自定义数据帧格式，其组成如表4所示。

表 4 蓝牙数据帧格式

字段	长度 (Byte)	说明
帧头 (STX)	1	0xAA
命令字	1	数据类型
数据长度	1	Data 长度
数据区	N	Pitch、Roll、Step
校验位	1	XOR 校验
帧尾 (ETX)	1	0x55

手机端接收到完整数据帧后，根据协议解析 Pitch 角、Roll 角及步数，并进行实时显示。

实际调试过程中，首先利用串口调试助手观察十六进制数据流，确认数据帧格式正确；随后完成手机端数据显示，实现智能手表与手机之间的数据同步。

3.3.10 计步算法设计

人体步行过程中，加速度会呈现明显的周期性变化，因此本项目采用基于加速度模长变化的阈值检测算法实现步数统计。

首先，根据三轴加速度计算当前加速度模长。

由于重力加速度始终存在，因此需要对加速度模长进行滤波处理，以减少高频噪声带来的影响。

系统设置上下两个阈值，当加速度模长由低于阈值上升至高于阈值，并再次下降时，判定完成一次完整步态周期，步数加一。

为了避免手部轻微晃动造成误计步，本系统增加了状态机判断以及最小时间间隔限制，只有满足完整步态变化且时间间隔满足要求时才增加步数。

计步结果实时保存在共享数据结构中，并同步显示于 OLED 和手机端。

3.3.11 任务同步与互斥保护

由于多个 FreeRTOS 任务需要同时访问系统数据，因此采用互斥锁（Mutex）保护共享资源。

Sensor Task 负责更新共享数据，包括姿态角、加速度、角速度、温度以及步数；Display Task 和 Bluetooth Task 只读取共享数据。

访问共享数据前，各任务首先申请互斥锁：

1. 获取 Mutex；
2. 复制共享数据到局部变量；
3. 释放 Mutex；
4. 使用局部变量完成后续处理。

这种设计能够有效避免多个任务同时访问共享变量导致的数据竞争，提高系统稳定性，同时减少互斥锁占用时间，不影响系统实时性能。

3.4 代码介绍

3.4.1 初始化和配置

系统上电后，首先完成硬件外设的初始化配置，主要包括以下函数调用：

- HAL_Init(); —— HAL 库底层初始化
- SystemClock_Config(); —— 系统时钟配置
- MX_GPIO_Init(); —— GPIO 引脚模式配置（编码器输入、按键、LED 等）
- MX_DMA_Init(); —— DMA 配置
- MX_I2C1_Init(); —— I2C1 初始化
- MX_I2C2_Init(); —— I2C2 初始化
- MX_USART2_UART_Init(); —— USART2 初始化
- MX_TIM2_Init(); —— 定时器 2 初始化
- MX_TIM3_Init(); —— 定时器 3 初始化
- MX_ADC1_Init(); —— ADC1 初始化
- MX_FREERTOS_Init(); —— FreeRTOS 初始化

3.4.2 主功能实现

系统启动 FreeRTOS 后，创建了以下五个任务，分别负责不同的功能模块：

taskTimeKeep 时间维护任务，每秒更新一次系统时间。

taskSensor 传感器数据处理任务，周期采集 MPU6050 数据并执行运动算法，具体包括：

- MPU6050_ReadAccel(); —— 读取三轴加速度数据
- MPU6050_CalcAngle(); —— 计算俯仰角和横滚角

taskMenu 菜单输入检测任务，扫描旋转编码器并产生菜单事件，具体包括：

- Encoder_Scan(); —— 检测编码器旋转方向（正转/反转）

- 按键消抖处理 (PB13)
- 发送菜单事件并显示

taskDisplay 界面显示任务，负责 OLED 屏幕刷新与菜单状态管理，具体包括：

- UI_DrawWatchFace(); ——绘制时间表盘界面
- UI_DrawMenu(); ——绘制多级菜单界面
- UI_DrawDataPage(); ——绘制传感器数据页面

taskBluetooth 蓝牙通信任务，周期性向手机端发送运动数据，主要调用：

- HAL_UART_Transmit(); ——通过 USART 发送数据

3.5 本章小结

本章介绍了智能手表软件系统的整体设计过程。系统采用 FreeRTOS 作为实时操作系统，将传感器采集、OLED 显示、菜单控制以及蓝牙通信划分为多个独立任务，并利用互斥锁实现共享数据保护。同时介绍了姿态解算、菜单控制、蓝牙通信协议以及计步算法等核心功能，为后续系统调试与测试奠定了软件基础。

4 功能展示

4.1 功能描述

本项目基于 STM32F103C8T6 实现智能手表功能原型系统, 通过各硬件模块与软件程序协同工作, 实现预期设计功能。

系统主要功能包括:

- 功能一: 时间显示功能;
- 功能二: 传感器数据显示功能;
- 功能三: 运动计步功能;
- 功能四: 菜单切换功能;
- 功能五: 蓝牙通信功能。

用户可以通过按键对系统进行操作, 实现不同功能之间的切换与控制。

4.2 演示结果

完成系统搭建后, 在面包板硬件平台上进行功能演示。通过运行程序, 验证各模块之间的协同工作情况。

实验演示结果如下:

- 显示内容正常、传感器数据能够正确读取。

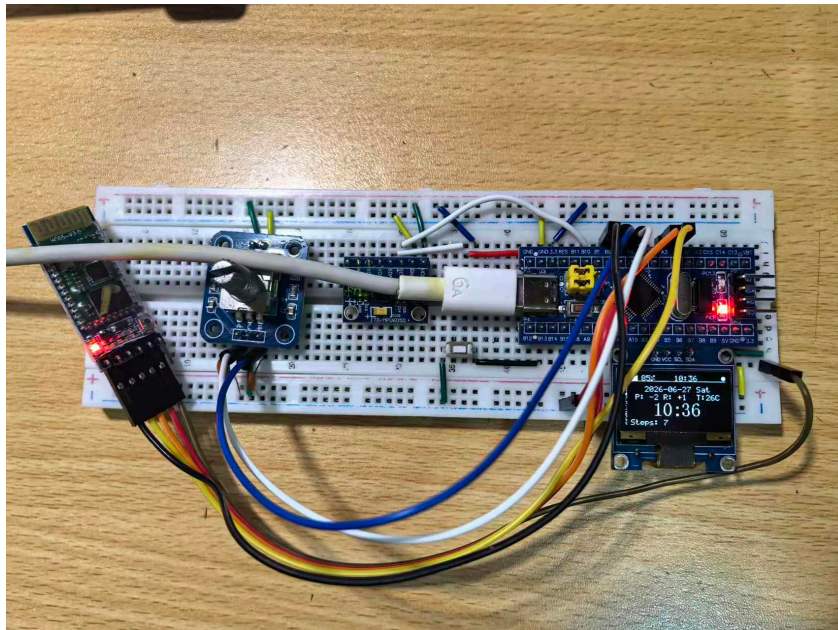


图 3 首页效果图

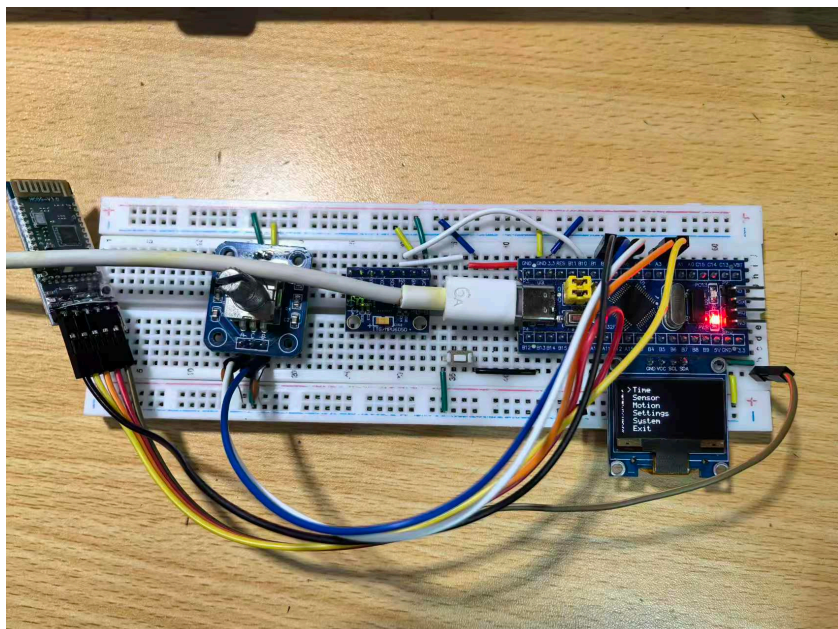


图 4 菜单选择效果图

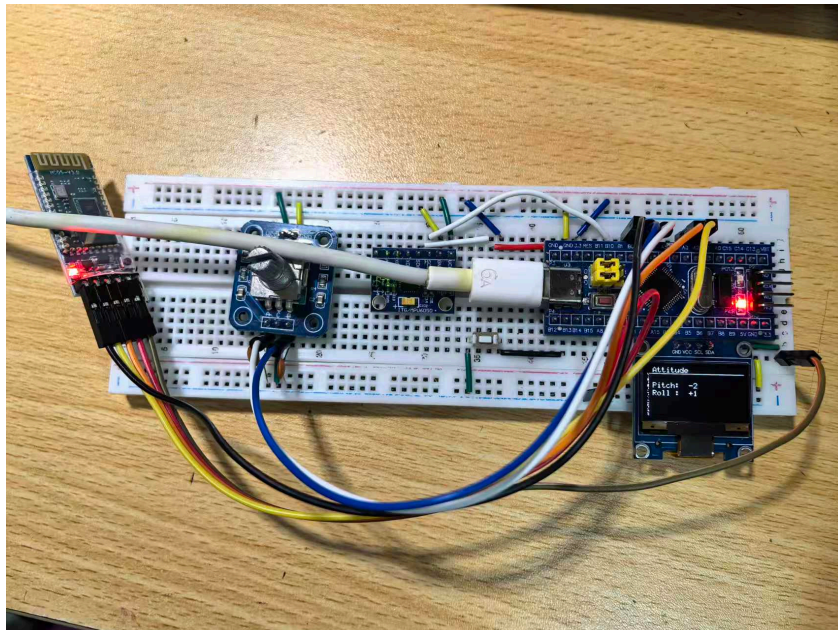


图 5 姿态角效果图

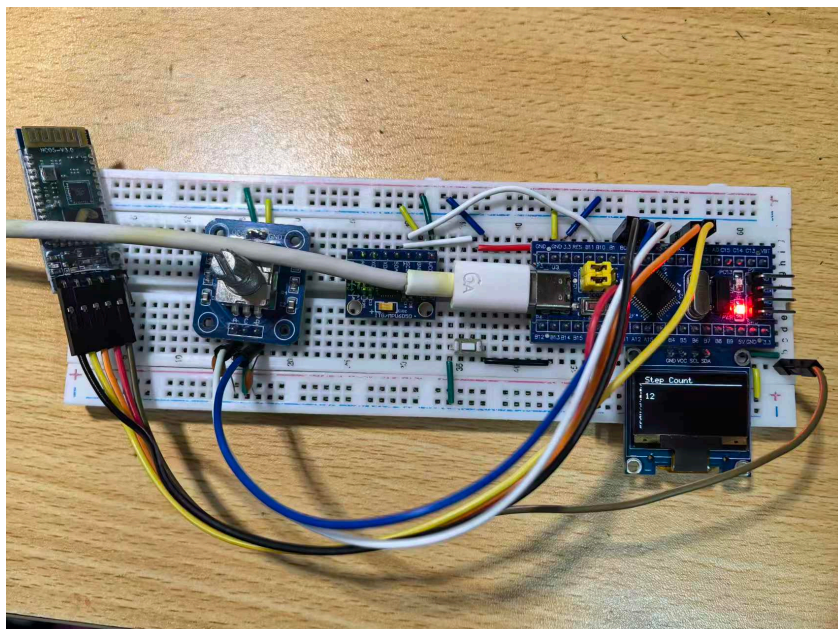


图 6 步数效果图

5 物料与成本

5.1 物料清单

本项目所使用的主要硬件器件如下表所示。

表 5 项目物料清单

序号	名称	型号/规格	数量	用途
1	主控芯片	STM32F103C8T6	1	系统核心控制单元
2	显示屏	0.96 寸 OLED (I2C 接口)	1	显示系统信息
3	陀螺仪模块	MPU6050	1	获取运动姿态数据
4	蓝牙模块	HC-05	1	实现无线通信功能
5	旋转编码器	旋转编码器	1	实现用户交互控制

5.2 项目总成本

项目所使用的硬件器件均为课程设计所需模块，部分器件已有库存，因此整体成本较低。

项目总成本如下：

表 6 项目成本统计

物料名称	数量	单价 (元)
STM32F103C8T6	1	6.83
OLED 显示屏	1	6.00
MPU6050 陀螺仪模块	1	15.13
HC-05 蓝牙模块	1	8.88
旋转编码器	1	4.2
项目总成本		41.04

5.3 损耗与损坏记录

项目开发过程中，各硬件模块均正常使用，未出现严重损坏情况。

6 个人贡献

6.1 报告撰写

负责项目报告的总体规划与撰写工作，包括：

- 完成项目背景分析、技术方案论证等理论部分
- 编写系统设计文档（硬件设计、软件架构）
- 整合全篇报告并进行最终排版优化

6.2 编码工作

- 完成 STM32 系统初始化代码（时钟配置、GPIO 初始化）
- 实现 mpu6050 的加速度读取与姿态角计算
- 实现蓝牙通信
- 共同完成 FreeRTOS 移植

6.3 硬件连接

设计并搭建完整的电路系统：

- STM32 核心板与 mpu6050、HC05 和 oled 的连接
- 旋转编码器的接入

6.4 开发问题描述与解决方案

系统采用逐步增加功能模块的方法进行开发，每完成一个模块后进行功能验证，再继续下一阶段开发，以降低系统调试难度。

6.4.1 FreeRTOS 移植调试

项目初期采用裸机程序开发，OLED 显示、MPU6050 姿态解算以及蓝牙通信均能够正常运行。

为了提高系统实时性能，后期将整个工程迁移至 FreeRTOS。

迁移完成后，工程能够正常编译下载，但是 OLED 仅完成初始化，界面无法刷新，各任务均未正常运行。

为了定位问题，采用逐步调试的方法进行排查。

首先，在各任务入口增加调试代码，确认任务是否成功创建。

随后，在 Display Task 中仅保留 OLED 刷新代码，验证 OLED 驱动本身不存在问题。

接着，将 Sensor Task 恢复为最简单的数据采集程序，发现 MPU6050 能够正常读取数据，说明 I2C 驱动工作正常。

最终发现问题主要来源于多个任务之间共享变量访问方式不正确，以及部分初始化代码仍保留裸机程序逻辑。

重新整理任务结构后，系统成功运行多个 FreeRTOS 任务。

6.4.2 MPU6050 姿态数据显示调试

完成 FreeRTOS 迁移后，首先恢复 MPU6050 数据采集任务。

系统周期读取三轴加速度、三轴角速度以及温度数据，并调用姿态解算算法计算 Pitch 角和 Roll 角。

调试过程中曾出现以下问题：

- Attitude_GetAngles 函数参数数量不一致；
- 结构体不存在 Yaw 成员；
- OLED 能够初始化，但姿态角始终不刷新；
- 共享数据更新失败。

通过重新检查函数声明、统一数据结构以及调整共享变量访问方式，上述问题均得到解决。

最终 OLED 能够实时显示 Pitch 角、Roll 角以及 IMU 各项数据。

6.4.3 菜单功能调试

菜单控制采用 TIM3 编码器模式。

系统初始设计为每检测到一次计数变化即翻页，但实际测试发现机械编码器存在一定抖动，导致菜单快速跳转。

为提高操作体验，对菜单算法进行了优化。

系统首先读取 TIM3 当前计数值，与上一时刻计数进行比较，当累计计数达到 4 时才切换一次菜单页面。

优化后菜单切换更加稳定，能够准确响应用户旋转操作。

6.4.4 蓝牙通信调试

蓝牙模块采用 HC-05，通过 USART1 与 STM32 连接，发送方式采用 DMA。

调试初期，手机串口助手无法正确显示数据，仅接收到异常字符。

随后依次检查：

- USART 波特率配置；
- HC-05 默认通信参数；
- DMA 发送是否正常；
- UART 发送完成回调；
- 蓝牙连接状态检测；
- 数据帧格式。

最终统一波特率为 38400bps，并采用 DMA 发送姿态数据。

系统设计的数据帧格式如下：

$$AA + CMD + LEN + DATA + CHECK + 55$$

手机串口助手能够稳定接收到完整的数据帧，例如：

```
AA 04 0C xx xx xx xx
xx xx xx xx
xx xx xx xx
CHK
55
```

进一步解析 DATA 区后，可正确获得 Pitch 角、Roll 角及运动步数，实现手机端实时数据显示。

7 项目成果与反思

7.1 成果展示

经过硬件电路搭建、软件系统开发以及多轮调试与优化，本项目成功完成了一款基于 STM32F103C8T6 的智能手表原型系统设计。系统采用 FreeRTOS 实时操作系统，实现了多任务协同运行，并完成了姿态检测、运动计步、OLED 显示、菜单交互以及蓝牙无线通信等功能，各模块能够稳定协同工作，达到预期设计目标。

在系统运行过程中，MPU6050 能够实时采集三轴加速度、三轴角速度及温度数据，通过姿态解算算法计算 Pitch 角和 Roll 角，并实时显示于 OLED 屏幕。基于加速度数据设计的计步算法能够完成人体步数统计，同时利用旋转编码器实现多页面菜单切换，提高了系统的人机交互体验。

此外，系统通过 HC-05 蓝牙模块实现与手机端的数据通信，能够将姿态角及运动步数等信息实时发送至手机进行显示和验证，验证了系统无线通信功能的可靠性。整个系统在 STM32F103C8T6 硬件平台上运行稳定，各任务能够按照预设周期完成数据采集、数据显示及数据发送等操作。

项目最终完成的主要成果如下：

- 完成基于 STM32F103C8T6 的智能手表硬件平台搭建，包括 MPU6050 姿态传感器、OLED 显示模块、HC-05 蓝牙模块及旋转编码器等外围电路；
- 实现基于 FreeRTOS 的多任务软件系统，完成传感器采集、OLED 显示、菜单控制及蓝牙通信等任务的设计与调度；
- 实现 MPU6050 姿态数据采集与姿态解算，能够实时显示 Pitch 角、Roll 角、三轴加速度、三轴角速度及温度信息；
- 实现基于加速度数据的运动计步功能，并同步显示于 OLED 和手机端；
- 实现旋转编码器菜单切换功能，支持多页面界面显示与用户交互；
- 实现基于 HC-05 蓝牙模块的数据传输，完成姿态角和步数等数据的实时无线发送；
- 完成系统整体联调与功能测试，验证了各功能模块能够稳定运行，满足课程设计的预期要求。

7.2 项目评估

7.2.1 成功之处

本项目按照预期完成了智能手表功能原型系统的设计与实现，主要成果如下：

- 整个项目采用模块化设计思想，以 STM32F103C8T6 作为主控制器，结合 MPU6050 六轴惯性传感器、OLED 显示模块、HC-05 蓝牙模块以及旋转编码器，实现了姿态检测、数据显示、蓝牙通信、菜单控制以及运动计步等功能。
- 采用 FreeRTOS 实时操作系统代替传统裸机程序；
- 加深了对嵌入式开发流程以及微控制器应用的理解。

7.2.2 不足之处

虽然本系统已经实现了智能手表的主要功能，但仍存在一定不足。

- 目前蓝牙通信采用 HC-05 经典蓝牙模块，仅实现串口透传功能，与现代智能手机的兼容性和扩展能力有限。若采用 BLE 低功耗蓝牙模块，可进一步提升设备的通信性能和续航能力。
- 目前系统采用 USB 供电，尚未完成锂电池供电、电池管理以及低功耗控制设计，因此距离实际智能手表产品仍存在一定差距。
- 当前 OLED 界面较为简单，仅实现基础数据显示，缺少动画效果、图标显示及更加丰富的人机交互界面，用户体验仍有提升空间。

7.3 个人反思

通过本次基于 STM32F103C8T6 智能手表系统的课程设计，我将课堂所学的嵌入式理论知识与实际工程开发相结合，对 STM32 开发流程有了更加深入的理解。从硬件搭建、外设驱动编写，到 FreeRTOS 多任务系统移植、姿态解算、OLED 显示、蓝牙通信以及计步算法实现，整个项目经历了反复调试和不断优化的过程。

在开发过程中，我遇到了任务调度、共享数据访问、蓝牙通信、传感器数据采集等多个问题。通过查阅资料、分析程序逻辑并不断进行调试，最终逐步解决了这些问题，也让我认识到嵌入式开发不仅需要扎实的理论基础，更需要耐心分析问题和不断实践的能力。

随着嵌入式技术和智能穿戴设备的不断发展，本系统仍具有较大的扩展空间。

首先，可进一步优化姿态融合算法，引入 Mahony、Madgwick 或扩展卡尔曼滤波等算法，提高姿态计算精度及系统稳定性。其次，可增加心率、血氧、体温等生理参数检测模块，实现更加丰富的健康监测功能，提升智能手表的应用价值。通信方面，可采用 BLE 蓝牙或 Wi-Fi 模块，实现与手机 App 的数据同步，并进一步扩展云端数据存储、远程监测等功能。此外，还可增加 RTC 实时时钟、电池管理系统、振动提醒、低功耗休眠、天气信息显示等功能，使系统更加接近实际智能手表产品。在人机交互方面，可设计更加丰富的图形界面，增加图标显示、菜单动画及多级菜单结构，提高系统易用性和美观程度。

综上所述，本课题完成了智能手表系统的基本设计目标，并为后续功能扩展和性能优化提供了良好的软件与硬件基础。今后我将继续深入学习嵌入式系统相关知识，不断提高自己的工程实践能力，为后续学习和开发打下更加坚实的基础。

A 设计图纸

电路原理图

完整电路原理图见附件。

B 代码清单

完整源代码见随附文件。